

AUTONOMOUS COMPLIANCE FABRIC

GRC Implementation Architecture

Deployment Reference for a 50-System Organisation

Classification	Public
Version	1.0 — Initial Architecture Draft
Audience	CISO, Head of Compliance, IT Infrastructure Lead
Scope	50-endpoint hybrid on-prem + cloud-read-only deployment

1. Executive Summary

This document defines the end-to-end implementation architecture for the Autonomous Compliance Fabric — a continuous, AI-driven GRC (Governance, Risk, and Compliance) platform designed for organisations operating in regulated industries. It is structured specifically for a 50-system deployment with zero-trust data handling principles.

The platform solves three problems that traditional GRC tooling cannot:

- Evidence gathering is manual, slow, and point-in-time. This platform collects evidence automatically every four hours.
- Compliance gaps are identified but remediation is left to the analyst. This platform generates the exact fix command — OS-specific — alongside the gap finding.
- Sensitive configuration data is sent to third-party cloud services. This platform keeps all raw data on-premise; only anonymised, cryptographically signed telemetry leaves the perimeter.

**DESIGN
RULE**

All raw configuration data is processed locally. The cloud dashboard receives only compliance scores, SHA-256 hashes, and AI-generated remediation commands. No hostnames, IP addresses, firewall rules, or user lists ever leave the organisation's network.

2. Problem Statement

Compliance in a regulated 50-system environment currently works as follows: a compliance analyst manually requests evidence from IT teams, collates screenshots and configuration exports, maps them to controls, identifies gaps, writes findings, and hands remediation to a sysadmin who must then research the correct fix. This cycle takes weeks per framework and produces a result that is already stale by the time it is reviewed.

Problem	Current State	What This Platform Delivers
Evidence collection	Manual — screenshots, emails, spreadsheets	Automated — agent-pulled config every 4 hours
Gap identification	Point-in-time — annual or quarterly	Continuous — live health score updated 4-hourly
Remediation guidance	Analyst researches fix manually	AI generates OS-specific CLI command at time of finding
Multi-framework mapping	Separate evidence per framework	One control mapped to all applicable frameworks automatically
Data residency	Config data sent to cloud GRC tools	All raw data processed locally — zero external exposure
Audit readiness	Evidence assembled on request — days of work	Dashboard exports signed, hashed evidence on demand

3. Architecture Overview

The platform consists of four logical zones. Each zone has a clearly defined trust boundary and a specific role. Nothing crosses a zone boundary without explicit cryptographic verification.

3.1 Zone Map

Zone	Location	Role	Trust Level
Zone A — Corporate LAN	On-premise	50 endpoints, Active Directory, syslog collector	Fully trusted — internal only
Zone B — Compliance Server	On-premise (dedicated VM)	Sentinel agents, local LLM, Qdrant, Vault, relay	Trusted — isolated VM, no inbound ports
Zone C — DMZ Relay	On-premise (org-controlled)	Firewall + forward proxy — outbound allowlist only	Semi-trusted — single egress rule
Zone D — Cloud Dashboard	Vendor SaaS (AWS)	Ingestion, signature verify, tenant store, CISO UI	Untrusted — read-only, no write-back path

3.2 Data Classification

Understanding what data sits where is the foundation of the security argument for this platform. The table below defines the data classification for every artefact the system handles.

Data Type	Classification	Where It Lives	Crosses Perimeter?
Raw firewall rules	SECRET	Zone A/B only	Never
Patch levels, port lists	CONFIDENTIAL	Zone A/B only	Never
User account lists	CONFIDENTIAL	Zone A/B only	Never
sshd_config, registry keys	CONFIDENTIAL	Zone A/B only	Never
SHA-256 hash of configs	INTERNAL	Zone B → Zone D	Yes — as proof of integrity
Control pass/fail scores	INTERNAL	Zone B → Zone D	Yes — anonymised
AI-generated remediation commands	INTERNAL	Zone B → Zone D	Yes — no org-specific data
Health score, gap count	INTERNAL	Zone D (cloud)	N/A — already in cloud

4. Hardware and System Requirements

These specifications are verified against the minimum requirements to run Mistral 7B in GGUF quantised format (Q4_K_M) for inference on CPU, Qdrant vector database, and the Python-based agent framework simultaneously on a single VM.

4.1 Compliance Server — Minimum Specification

Component	Minimum	Recommended	Rationale
CPU	8 vCPUs (Intel Xeon E-series or AMD EPYC)	16 vCPUs	Mistral 7B Q4_K_M requires ~6 threads for acceptable inference. Remaining cores run agents and Qdrant.
RAM	32 GB DDR4 ECC	64 GB	Mistral 7B Q4_K_M uses ~6 GB model weight in RAM. 8 GB for OS and agents. 16 GB headroom for Qdrant + concurrent scans.
Storage	500 GB NVMe SSD	1 TB NVMe SSD	Model weights: ~4 GB. Framework embeddings (Qdrant): ~20 GB. Evidence log (append-only): ~50 GB/year for 50 endpoints.
Network	1 Gbps NIC (dual port)	10 Gbps NIC	Port 1: internal LAN (read-only agent traffic). Port 2: DMZ-only, single outbound TLS 1.3 connection.
GPU	Not required	NVIDIA RTX 4090 (24 GB VRAM)	Optional. Reduces LLM inference from ~45 sec/query to ~3 sec. Not needed for 4-hourly batch scans.
OS	Ubuntu 22.04 LTS Server (headless)	Same	LTS until 2027. Hardened via CIS Benchmark Level 2. No desktop environment installed.
Virtualisation	VMware ESXi 8 or Proxmox VE 8	Same	Either works. Pin CPU cores — no NUMA balancing — to ensure LLM inference performance is predictable.

4.2 Endpoint Requirements (All 50 Systems)

No agent software is installed on endpoints. The Sentinel agents run on the Compliance Server and connect to endpoints remotely. The only requirement on each endpoint is that a read-only service account exists and the appropriate remote management protocol is enabled.

Requirement	Windows Endpoints	Linux Endpoints
Remote protocol	WMI over DCOM (port 135 + dynamic RPC)	SSH (port 22, key-based auth only)
Service account	Domain account with 'Remote Management Users' group membership only	Local or LDAP account — sudo NOPASSWD denied, read-only shell access only
Firewall rule	Allow inbound WMI from Compliance Server IP only	Allow inbound SSH from Compliance Server IP only

Requirement	Windows Endpoints	Linux Endpoints
Authentication	Kerberos (domain-joined) or NTLM fallback	Ed25519 SSH key pair — private key stored in HashiCorp Vault
Minimum OS	Windows 10 Pro / Windows Server 2016	Ubuntu 20.04, RHEL 8, Debian 11 or later
GPO/Config change	WMI namespace permissions — CIMV2 read-only for service account	sshd_config: AllowUsers compliance-svc, PasswordAuthentication no

5. Software Stack

Every component below is open-source or available under a commercial licence that permits on-premise deployment with no data egress requirements. Versions listed are the minimum tested stable releases as of 2025.

Component	Software	Version	Licence	Purpose
LLM runtime	llama.cpp (GGUF format)	b3447+	MIT	Runs Mistral 7B locally without GPU requirement
LLM model	Mistral 7B Instruct Q4_K_M	v0.3	Apache 2.0	Policy mapping, gap analysis, remediation generation
Vector database	Qdrant	1.9+	Apache 2.0	Stores framework embeddings for RAG — ISO, NIST, PCI DSS
Agent framework	Python + LangChain	3.11 / 0.2+	MIT	Orchestrates sentinel agents and LLM calls
Key management	HashiCorp Vault	1.16+	BSL 1.1	Stores Ed25519 private keys, SSH keys — never written to disk
Payload signing	Ed25519 via PyNaCl	1.5+	Apache 2.0	Signs outbound telemetry — verifiable by cloud without sharing key
WMI client (Win)	impacket / wmiexec	0.12+	Apache 2.0	Queries Windows endpoints from Linux compliance server
SSH client (Linux)	Paramiko	3.4+	LGPL 2.1	Connects to Linux endpoints — key auth only via Vault
API layer	FastAPI + Uvicorn	0.111+	MIT	Internal REST API — compliance server to local dashboard or DMZ relay
Scheduler	APScheduler	3.10+	MIT	Triggers scans every 4 hours — configurable
Cloud ingestion	AWS Lambda + API Gateway	N/A	Commercial	Receives signed telemetry, verifies signature, writes to tenant store
Tenant key store	AWS KMS	N/A	Commercial	Per-tenant encryption key — cross-tenant reads impossible
Dashboard	React + Recharts	18 / 2.x	MIT	Read-only CISO-facing UI — no write-back endpoints

6. Step-by-Step Implementation

The following eight steps represent the complete operational cycle from endpoint poll to CISO dashboard. Each step includes what happens, how it differs between Windows and Linux endpoints, and — critically — why this specific approach was chosen over simpler alternatives.

STEP 01	Infrastructure preparation and network segmentation Deploy the Compliance Server as an isolated VM on your existing hypervisor (VMware ESXi or Proxmox). Assign it a static IP on the corporate LAN segment. Install Ubuntu 22.04 LTS Server — no GUI. Apply CIS Benchmark Level 2 hardening immediately: disable unused services, enforce UFW firewall, configure auditd logging. Create two network interfaces: eth0 on the internal LAN (for reaching endpoints), eth1 on the DMZ segment (for the single outbound TLS connection to the cloud). Configure UFW on the Compliance Server: inbound — allow only SSH from admin workstation IP. Outbound — allow only HTTPS/443 to the cloud endpoint hostname from eth1. On the corporate firewall: add one rule — allow TCP/443 outbound from Compliance Server DMZ IP to cloud.yourcompliance.com. Deny all other outbound from this host. WHY: Network segmentation defines your blast radius. If the Compliance Server is ever compromised, the attacker has access only to the read-only service account on endpoints and a single outbound HTTPS channel. No inbound firewall rules means no inbound attack surface. The dual-NIC design ensures internal scan traffic and outbound telemetry never share the same interface — a simple but effective separation that your firewall team can audit in five minutes.
STEP 02	Service account creation — Windows and Linux WINDOWS: In Active Directory, create a dedicated account named 'svc-compliance'. Add it to 'Remote Management Users' and 'Performance Monitor Users' groups only. Grant WMI namespace CIMV2 read-only permissions via WMI Control > Security. Do not add to Domain Admins, Local Admins, or any privileged group. Deploy via GPO to all Windows endpoints. LINUX: Generate an Ed25519 SSH key pair on the Compliance Server using <code>ssh-keygen -t ed25519</code>. Store the private key in HashiCorp Vault (transit secrets engine — key never written to disk). Deploy the public key to <code>/home/compliance-svc/.ssh/authorized_keys</code> on all Linux endpoints via Ansible or your existing configuration management tool. LINUX sshd_config additions on each endpoint: <code>AllowUsers compliance-svc</code>, <code>PasswordAuthentication no</code>, <code>PubkeyAuthentication yes</code>, <code>PermitRootLogin no</code>. Test both accounts immediately: confirm WMI query returns data, confirm SSH key auth works, confirm neither account can write, delete, or escalate privileges. WHY: The read-only, least-privilege service account is the single most important security control in this deployment. It is not a convenience — it is the constraint that makes the entire architecture acceptable to a security-conscious organisation. If an analyst asks why you chose WMI over PowerShell remoting for Windows, the answer is that WMI queries are read-only by design and do not permit command execution on the remote host. For Linux, key-based SSH with a locked-down <code>sshd_config</code> means even if the private key is extracted from Vault, it cannot be used to escalate or pivot without a second compromise.
STEP 03	Compliance Server — software installation and LLM setup Install <code>llama.cpp</code> from source on the Compliance Server. Download Mistral 7B Instruct Q4_K_M in GGUF format (~4.1 GB). Verify SHA-256 checksum against the official Hugging Face release hash before loading.

Run inference performance test: `llama-cli -m mistral-7b-instruct-v0.3.Q4_K_M.gguf -p 'What is ISO 27001 control A.9.4.3?' -n 200`. Baseline: expect 8-15 tokens/second on 8 vCPUs without GPU. For a 4-hourly compliance scan of 50 endpoints, this is sufficient.

Install Qdrant as a Docker container on the Compliance Server (Docker on Ubuntu 22.04). Map port 6333 to localhost only — not exposed on any network interface.

Load framework embeddings: download ISO 27001:2022, NIST 800-53 Rev 5, PCI DSS v4.0 control text. Chunk, embed using a local sentence-transformer model, and index in Qdrant. This is a one-time operation taking approximately 20-30 minutes.

Install HashiCorp Vault in dev mode initially for testing, then switch to production mode with an encrypted file storage backend. Initialise, unseal, store the Ed25519 private key and the SSH private key in the transit secrets engine.

Install Python 3.11, create a virtualenv, install: langchain, impacket, paramiko, qdrant-client, pynacl, fastapi, uvicorn, apscheduler.

WHY: Mistral 7B Q4_K_M was specifically chosen over larger models (Llama 3 70B, Mixtral 8x7B) because it runs entirely in CPU RAM within the 32 GB specification. The Q4_K_M quantisation format gives approximately 90% of the full-precision model quality at 25% of the memory footprint — verified by independent benchmarks on compliance-domain text tasks. Qdrant was chosen over ChromaDB because it supports persistent storage with HNSW indexing, meaning framework embeddings survive a server restart without re-embedding. HashiCorp Vault is the only production-grade secret store that provides a transit secrets engine — the private key literally never exists as a file on disk. This is the difference between enterprise-grade key management and storing a key in a config file.

Sentinel agents — endpoint polling

STEP 04

WINDOWS AGENT: Uses impacket's WMI client to query each Windows endpoint. Collected data includes: Win32_QuickFixEngineering (patches), Win32_UserAccount (local accounts), NetFirewallRule via PowerShell registry query (read-only), Win32_LogicalDisk (encryption status via BitLocker WMI class), installed software via Win32_Product.

LINUX AGENT: Uses Paramiko SSH client with the Vault-stored Ed25519 key to connect to each Linux endpoint. Runs the following read-only commands: `cat /etc/ssh/sshd_config`, `ss -tlnp` (open ports), `cat /etc/passwd` (user accounts), `dpkg -l` or `rpm -qa` (installed packages), `systemctl list-units --state=running`, `ufw status verbose` or `iptables -L -n`.

Both agents write raw collected data to an encrypted local store on the Compliance Server (AES-256-GCM, key derived from Vault). This store is never read by the outbound relay — it is the input to Step 5 only.

Scan schedule: APScheduler triggers both agents every 4 hours. Each full scan of 50 endpoints completes in under 10 minutes (parallel threads, 5 concurrent connections per agent type).

Failed connections are logged with timestamp and reason. Three consecutive failures for any endpoint raise a 'connectivity alert' that appears on both the local dashboard and the cloud health score as a degraded control.

WHY: The decision to use WMI for Windows and SSH for Linux reflects the actual security posture of each platform, not convenience. WMI is the Microsoft-endorsed remote management interface and is auditable via Windows Event Log (Event IDs 4624, 4672). PowerShell remoting (WinRM) was deliberately excluded because it supports command execution, not just data retrieval. On Linux, SSH key auth with a restricted shell is the standard read-only remote access pattern — it is already in use in most organisations for monitoring. The agent runs from the Compliance Server outward; it does not install anything on endpoints, which eliminates the endpoint agent deployment and maintenance problem entirely.

Local LLM analysis and OS-specific remediation generation

STEP 05

The collected raw data from Step 4 is fed to the LangChain orchestration layer. For each endpoint, the LLM receives a structured prompt: the collected configuration data, the relevant framework controls

(retrieved from Qdrant via RAG), and an instruction to return: (1) pass/fail per control, (2) the specific configuration gap, (3) the exact remediation command for the detected OS.

WINDOWS REMEDIATION EXAMPLE — If BitLocker encryption is not enabled on C drive: 'Enable-BitLocker -MountPoint C: -EncryptionMethod Aes256 -UsedSpaceOnly -TpmProtector' (PowerShell, run as Administrator).

LINUX REMEDIATION EXAMPLE — If SSH PasswordAuthentication is set to yes: 'sudo sed -i s/PasswordAuthentication yes/PasswordAuthentication no/ /etc/ssh/sshd_config && sudo systemctl restart sshd'

LINUX REMEDIATION EXAMPLE — If UFW is inactive: 'sudo ufw enable && sudo ufw default deny incoming && sudo ufw default allow outgoing'

The LLM is also instructed to perform cross-framework mapping: if 'SSH key authentication enforced' satisfies both ISO 27001 A.9.4.3 and NIST AC-17(2), it marks both controls as satisfied from a single evidence item. This is the deduplication engine.

Output from this step: a structured JSON object per endpoint containing control IDs, pass/fail status, evidence hash, gap description, and OS-specific remediation command.

WHY: Generating the remediation command at the time of gap detection — rather than leaving it to the analyst — is the single most operationally significant feature of this platform. The alternative is the analyst receives a finding like 'SSH password authentication is enabled' and must then research the correct command for the specific OS version, test it in a non-production environment, and write a change request. That process takes days. The AI delivers a copy-pasteable, OS-version-aware command alongside the finding. The analyst's job shifts from researcher to verifier — a fundamentally lower-risk and faster role. The cross-framework deduplication is equally important: without it, a 50-system organisation managing three frameworks would maintain three separate evidence sets for the same underlying controls.

Data normalisation and anonymisation

Before any data moves toward the outbound relay, the Normaliser module strips all organisation-identifying information. What is removed: hostnames, IP addresses, usernames, domain names, raw configuration file content, software version strings that could fingerprint the environment.

What is retained: SHA-256 hash of each configuration file (proves the config was reviewed without revealing its content), pass/fail status per control ID, gap description in generic terms (e.g., 'SSH password authentication enabled' not 'Server DC-PROD-01 has PasswordAuthentication yes'), OS-specific remediation command (generic — does not reference specific hostnames), framework health score per control family.

STEP 06

The Normaliser also creates a diff from the previous scan: only changed control statuses are included in the outbound payload. An endpoint that passed all controls in the last scan and still passes them generates no outbound data for those controls — reducing payload size and cloud storage requirements.

Output payload size for a 50-endpoint organisation with typical configuration: approximately 40-80 KB per scan cycle, compressed. This crosses the DMZ relay as a single HTTPS POST request.

WHY: This step is the legal and compliance foundation of the architecture. For a bank or financial institution, sending configuration data to a third-party cloud service — even an encrypted one — may trigger data residency obligations, regulatory notification requirements, or contractual obligations to clients. By anonymising at this layer, the data that leaves the perimeter is legally equivalent to a compliance score, not a configuration disclosure. This argument needs to be prepared and documented for your legal and DPO teams before the platform goes live. The SHA-256 hash approach is industry-standard: it is used by software supply chain tools (SLSA, SBOM) and is accepted as cryptographic evidence in audit contexts.

Cryptographic signing, encryption, and outbound relay

STEP 07

The Outbound Relay agent retrieves the anonymised payload from the Normaliser. It requests a signing operation from HashiCorp Vault's transit engine — the payload is signed using Ed25519 without the private key ever leaving Vault. The signature is appended to the payload.

The signed payload is then encrypted using AES-256-GCM with a session key negotiated fresh for each transmission via TLS 1.3. The TLS connection terminates at the DMZ proxy, which re-establishes TLS to the cloud endpoint — this is intentional and required by the firewall allowlist model.

Anti-replay protection: a UUID nonce and Unix timestamp (± 5 minute tolerance) are included in every payload. The cloud ingestion endpoint rejects any payload with a previously seen nonce or a timestamp outside the tolerance window.

The DMZ forward proxy is configured with a single allowlist rule: CONNECT requests to cloud.yourcompliance.com port 443 only. All other CONNECT requests are denied. The proxy logs every connection attempt — this log should feed into your existing SIEM.

Transmission frequency: every 4 hours, aligned to the scan schedule. If the cloud endpoint is unreachable, the relay retries with exponential backoff (2, 4, 8, 16 minutes). After 5 failures, it logs a critical alert and sends an internal notification to the compliance team via the local dashboard. It does not drop the payload — it queues it for the next successful connection.

WHY: Ed25519 was chosen over RSA-2048 for payload signing because it produces smaller signatures (64 bytes vs 256 bytes), is faster to generate and verify, and has a stronger security margin against advances in cryptanalysis. The decision to keep the private key inside Vault's transit engine — rather than reading it into memory and signing in Python — is critical: if the Compliance Server process is memory-dumped by an attacker, the private key is not in the process heap. AES-256-GCM was chosen over AES-256-CBC because GCM provides authenticated encryption — it simultaneously encrypts and provides an authentication tag, meaning a tampered ciphertext is detected before decryption. These are not academic distinctions; they are the choices your security team and external auditors will ask about.

Cloud ingestion, tenant storage, and CISO dashboard

The cloud ingestion endpoint (AWS API Gateway + Lambda) receives the payload. The Lambda function retrieves the organisation's registered Ed25519 public key from AWS Secrets Manager and verifies the signature. If verification fails for any reason — tampered payload, wrong key, replay — the request is rejected with HTTP 401 and an alert is logged.

Verified payloads are decrypted and written to the organisation's isolated data partition. Each organisation has its own AWS KMS customer-managed key (CMK). Data is encrypted at rest using the CMK. The application layer never sees plaintext data from another tenant — isolation is enforced at the key management layer, not just the application layer.

Data is written append-only. Historical compliance states are retained for a configurable period (default: 12 months). This provides trend analysis: 'did the access control score improve after the August patching cycle?'

The CISO dashboard API exposes GET endpoints only. There are no POST, PUT, PATCH, or DELETE endpoints accessible to dashboard users. An attacker who compromises a dashboard session token has read access to compliance scores and gap findings — they cannot modify data, trigger scans, or reach back into the on-premise environment.

Dashboard views: (1) Overall health score — single number, 0-100, updated every 4 hours. (2) Framework breakdown — score per framework (ISO 27001, NIST, PCI DSS). (3) Gap list — each gap with OS-specific remediation command, copyable. (4) Trend chart — health score over 30/90/365 days. (5) Evidence log — hashed evidence items with timestamps, exportable as signed PDF for auditors.

External auditor access: time-boxed, read-only tokens issued via the dashboard. Scope is limited to specific framework controls and a specific time window. Token expiry is enforced server-side — no refresh tokens.

WHY: The read-only cloud dashboard is the only component visible to people outside the organisation's IT function. Its design — GET-only API, no write-back path, time-boxed auditor tokens — means that even a full compromise of the cloud layer gives an attacker nothing useful for attacking the on-premise environment. The cloud is a reporting surface, not a control plane. This is the architectural decision that makes the platform acceptable to security-conscious organisations that would otherwise refuse to connect any compliance tooling to a third-party cloud service.

STEP 08

7. Windows vs. Linux — Implementation Differences

The Sentinel agent logic branches at the OS detection level. Both branches connect from the Compliance Server to the endpoint; neither installs software on the endpoint. The differences are in protocol, authentication, and what data is collected.

7.1 Side-by-side comparison

Dimension	Windows Endpoints	Linux Endpoints
Protocol	WMI over DCOM (port 135 + RPC dynamic ports 49152-65535)	SSH (port 22) — key authentication only
Auth mechanism	Kerberos (domain-joined) — service account in 'Remote Management Users' group	Ed25519 SSH key — private key in HashiCorp Vault, never on disk
Python library	impacket (WMI client) — actively maintained, used in enterprise security tooling	Paramiko (SSH client) — standard Python SSH library
Patch check	Win32_QuickFixEngineering WMI class — returns KB IDs and install dates	dpkg -l (Debian/Ubuntu) or rpm -qa --queryformat (RHEL/CentOS)
Open ports	NetTCPConnection WMI class or registry-based firewall query	ss -tlnp — socket statistics, no root required for read
User accounts	Win32_UserAccount WMI class — local accounts only; domain accounts via LDAP separately	cat /etc/passwd and getent passwd — local and LDAP-synced accounts
Encryption check	BitLocker status via Win32_EncryptableVolume WMI class (requires CIMV2\Security\MicrosoftVolumeEncryption namespace)	dmsetup status or cryptsetup status — checks LUKS encryption
Service/process check	Win32_Service WMI class — name, state, start mode, path	systemctl list-units --type=service --state=running
Firewall rules	Registry read: HKLM\SYSTEM\CurrentControlSet\Services\SharedAccess\Parameters\FirewallPolicy	ufw status verbose (Ubuntu) or iptables -L -n (RHEL) — read-only
SSH config check	N/A — Windows does not use sshd by default	cat /etc/ssh/sshd_config — critical for access control controls
Remediation format	PowerShell commands (Windows Server 2019/2022 compatible)	Bash commands with systemctl, sed,

Dimension	Windows Endpoints	Linux Endpoints
		ufw, apt/yum — OS-version aware
Logging	Windows Event Log — WMI access logged under Event ID 4624 (logon)	Linux auth.log / journald — SSH login and command execution logged
GPO requirement	WMI namespace permissions deployed via Group Policy	sshd_config change + authorized_keys — deployed via Ansible or Chef

7.2 Remediation command examples — by OS and control

The following table shows representative remediation commands generated by the local LLM for common compliance gaps. These are the exact commands surfaced in the CISO dashboard for administrator action.

Control (Framework)	Gap Found	Windows Remediation	Linux Remediation
Password complexity (NIST IA-5)	Min length < 14 characters	net accounts /MINPWLEN:14 (CMD, run as Admin)	sudo sed -i 's/minlen=.*minlen=14/' /etc/security/pwquality.conf
Disk encryption (ISO A.10.1.1)	Drive not encrypted at rest	Enable-BitLocker -MountPoint C: -EncryptionMethod Aes256 -UsedSpaceOnly -TpmProtector	sudo cryptsetup luksFormat /dev/sdX && sudo cryptsetup luksOpen (new deployments only — note: requires reformat)
SSH hardening (ISO A.9.4.3)	Password auth enabled	N/A (Windows OpenSSH optional — check only if installed)	sudo sed -i 's/PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && sudo systemctl restart sshd
Firewall active (NIST SC-7)	Host firewall disabled	Set-NetFirewallProfile -Profile Domain,Public,Private -Enabled True (PowerShell, Admin)	sudo ufw enable && sudo ufw default deny incoming && sudo ufw default allow outgoing
Auto-update enabled (PCI 6.3.3)	Automatic updates off	Set-ItemProperty 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU' -Name 'AUOptions' -Value 4	sudo apt-get install unattended-upgrades -y && sudo dpkg-reconfigure -plow unattended-upgrades
Audit logging (NIST AU-2)	Audit policy not configured	auditpol /set /category:'Logon/Logoff' /success:enable /failure:enable	sudo systemctl enable auditd && sudo systemctl start auditd && sudo auditctl -e 1

IMPORTANT

All remediation commands must be reviewed by a qualified system administrator before execution. The AI generates commands based on OS type and version detected during



the scan. Commands should be tested in a non-production environment first. Change management procedures apply — a remediation command is not a substitute for a formal change request.

8. Security Model

This section documents the security assumptions, controls, and residual risks of the architecture. It is intended to be shared with your security team and used as the basis for a threat model review before deployment.

8.1 Trust assumptions

- The Compliance Server VM is assumed to be running on a trusted hypervisor managed by the organisation's own IT team.
- The internal network between the Compliance Server and endpoints is assumed to be protected by the organisation's existing network security controls (VLANs, 802.1X, IDS).
- HashiCorp Vault is assumed to be correctly initialised, sealed when not in use, and accessible only from the Compliance Server process.
- The cloud dashboard vendor (AWS) is assumed to correctly implement KMS key isolation and to have no access to the organisation's CMK.

8.2 Threat model — key risks and mitigations

Threat	Likelihood	Impact	Mitigation
Compliance Server compromised	Low	High	Attacker gains read-only access to endpoints and a signing key in Vault. Vault can be revoked immediately. No write access to endpoints. No access to cloud credentials.
Telemetry payload intercepted in transit	Low	Low	Payload is AES-256-GCM encrypted and Ed25519 signed. Interception yields ciphertext with no decryption key. Signature prevents tampering.
Replay attack on cloud ingestion	Low	Medium	UUID nonce + ± 5 min timestamp window prevents replay. Cloud rejects any previously seen nonce.
Cloud tenant data breach	Low	Low	Data contains only hashes and scores — no raw configs. CMK isolation prevents cross-tenant reads. No organisational infrastructure data is actionable from cloud data alone.
Read-only service account privilege escalation (Windows)	Low	High	Mitigated by WMI namespace restriction to CIMV2 read-only. Service account not in any admin group. Regular review of AD group memberships.
LLM generates incorrect remediation command	Medium	Medium	Remediation commands are labelled as AI-generated and require administrator review before execution. Change management process applies. No auto-execution capability exists in the platform.
Framework embeddings become outdated	Medium	Medium	Qdrant embeddings are versioned. New framework versions trigger an update alert. Re-embedding is a scheduled maintenance task, not a continuous process.

9. Implementation Timeline

This timeline assumes a team of two: one infrastructure engineer and one compliance/security analyst. It is based on realistic working estimates for each task including testing time.

Phase	Tasks	Owner	Duration
Phase 1: Infrastructure	Deploy Compliance Server VM, apply CIS hardening, configure dual-NIC network, set firewall rules on corporate perimeter	Infrastructure Engineer	2-3 days
Phase 2: Service accounts	Create AD service account, configure WMI namespace permissions, deploy via GPO, generate SSH keys, deploy to Linux endpoints, test both connections	Infrastructure Engineer	1-2 days
Phase 3: LLM setup	Install llama.cpp, download and verify Mistral 7B GGUF, run inference tests, install Qdrant, load ISO 27001 / NIST / PCI DSS embeddings	Infrastructure Engineer	1-2 days
Phase 4: Vault & signing	Install and configure HashiCorp Vault, store Ed25519 and SSH private keys in transit engine, test signing operations	Infrastructure Engineer + Security Analyst	1 day
Phase 5: Agent development	Build and test Windows WMI agent, build and test Linux SSH agent, build Normaliser, build Outbound Relay, integration test with 3-5 pilot endpoints	Security Analyst (Python)	5-7 days
Phase 6: Cloud setup	Provision AWS API Gateway + Lambda ingestion, configure KMS CMK per tenant, build React dashboard, end-to-end test	Infrastructure Engineer	3-4 days
Phase 7: Pilot run	Run first full scan of all 50 endpoints, review AI output for accuracy, validate remediation commands against test machines, review dashboard output with CISO	Both	2-3 days
Phase 8: Go live	Enable production schedule (4-hourly), document runbook, brief CISO and compliance team, establish false-positive review process	Security Analyst	1 day
Total			16-23 working days (3-5 weeks)

10. Recommendations for Improvement

The architecture documented in this paper is production-viable as described. The following recommendations represent the next iteration — features and controls that are not required for initial deployment but should be planned for in the 6-12 month roadmap.

10.1 Role-based access control (Priority: High — implement before go-live)

GAP

Neither the on-premise local dashboard nor the cloud dashboard currently defines who can view what, who can approve what, or who can export evidence. This is not optional for a regulated environment.

Define the following roles before presenting to the CISO:

- Compliance Officer — can view all findings, approve false-positive exceptions, export audit reports. Cannot trigger scans manually.
- CISO — read-only view of all findings, health scores, trend data. Cannot approve exceptions (separation of duties).
- System Administrator — can view only the remediation commands for their assigned endpoints. Cannot see other endpoints' findings.
- External Auditor — time-boxed, framework-scoped, read-only token. Automatically expires. No access to remediation commands.
- Platform Administrator — can configure scan schedules, add or remove endpoints, update framework embeddings. Cannot view finding content (separation of duties).

Technical implementation: JWT-based RBAC in the FastAPI layer, with roles stored in Vault and synchronised to the cloud dashboard via a separate, authenticated API. Estimated implementation: 3-5 additional days.

10.2 Change management workflow (Priority: High — implement in first 3 months)

GAP

The platform currently surfaces a remediation command. There is no process to control what happens next. In a regulated environment, applying a change to a production system without a formal change request, approval, and post-change verification is itself a compliance violation.

Implement the following workflow for each remediation command:

1. AI generates remediation command — appears in dashboard as 'Pending Review'.
2. **Step 2:** Compliance Officer reviews and approves — command moves to 'Approved, Awaiting Execution'.
3. **Step 3:** Auto-create change ticket in Jira or ServiceNow via API integration.
4. **Step 4:** System Administrator executes command and marks ticket complete.
5. **Step 5:** Sentinel agent re-polls the endpoint at next scan cycle — verifies the control now passes.
6. **Step 6:** Evidence of fix (hash of new config, timestamp, approver name) is appended to the audit log.

This workflow turns the platform from a detector into a closed-loop remediation system. It is also the artefact your auditor will ask for: 'show me that a gap was found, reviewed, approved, fixed, and verified.' Without this workflow, that evidence must still be assembled manually.

10.3 False-positive and exception management (Priority: Medium — implement in first 3 months)

GAP

The local LLM will occasionally flag a control as failed when a compensating control satisfies the requirement. Without a mechanism to record this, the health score is permanently inaccurate and auditors lose trust in the platform.

Add an 'Accept Risk' or 'Mark as Exception' function to each finding:

- Compliance Officer can mark a finding as an accepted exception with a business justification, an approver, and an expiry date.
- Exceptions appear on the dashboard with a distinct status — they do not contribute to the 'failed' count but are visible and auditable.
- Exceptions expire automatically. On expiry, the control reverts to its AI-assessed status. The Compliance Officer receives a notification 30 days before expiry.
- All exceptions are included in audit export — an auditor can see every exception, its justification, and who approved it.

10.4 SIEM integration (Priority: Medium — implement in first 6 months)

Compliance findings should flow into the organisation's existing SIEM (Splunk, QRadar, Microsoft Sentinel) as structured events. This allows:

- Correlation between a compliance gap and a security incident — e.g., 'this system had host firewall disabled (compliance gap) three days before the lateral movement was detected (security event)'.
- Compliance health score as a SIEM dashboard panel — no need to log into a separate compliance platform for a quick status check.
- Alert on health score degradation below a threshold — e.g., PCI DSS score drops below 80 triggers a SIEM alert to the SOC.

Implementation: FastAPI webhook endpoint on the Compliance Server pushes findings to SIEM in CEF or JSON format. One-directional — SIEM receives data, cannot trigger actions on the Compliance Server.

10.5 HSM for key storage (Priority: Medium — high-security environments)

HashiCorp Vault's transit secrets engine is production-grade for most organisations. For environments with the highest security requirements (banking core systems, critical national infrastructure), replace Vault with a dedicated Hardware Security Module (HSM):

- Thales Luna Network HSM or AWS CloudHSM — both FIPS 140-2 Level 3 certified.
- The Ed25519 private key is generated inside the HSM and physically cannot be extracted — even by the HSM administrator.
- All signing operations are performed inside the HSM tamper-resistant boundary.

- Cost: Thales Luna is approximately USD 20,000-40,000 per appliance. AWS CloudHSM is approximately USD 1.45/hour per HSM partition.

For most 50-system deployments in the financial sector, HashiCorp Vault is sufficient. An HSM is recommended if the organisation is subject to FIPS 140-2 Level 3 requirements or handles payment card data at a volume that attracts PCI DSS SAQ D or higher.

10.6 Multi-framework auto-update (Priority: Low — 12-month roadmap)

When a new regulatory framework is published (e.g., DORA technical standards, UAE IA Regulation updates, SAMA CSF revision), the current architecture requires a manual re-embedding process. In future iterations:

- Subscribe to official framework RSS feeds or regulatory body APIs where available.
- When a new version is detected, automatically download, chunk, embed, and index the new framework in Qdrant.
- Flag any controls in the new version that differ from the previous version and re-assess affected endpoints at the next scan cycle.
- Notify the Compliance Officer of the new framework version and the delta in control requirements.

This feature requires significant validation before automation — regulatory text is ambiguous and the mapping from text to technical control requires human review. The recommended approach is semi-automated: the system flags the change, the Compliance Officer reviews the mapping, and then approves the update.

10.7 Offline / disconnected operation mode

The current architecture assumes the DMZ relay can reach the cloud dashboard every 4 hours. For environments that go into periodic network isolation (incident response lockdown, scheduled maintenance windows, disaster recovery scenarios):

- Queue telemetry payloads locally during the isolation period — encrypted and signed as normal.
- On reconnection, transmit queued payloads in chronological order. The cloud ingestion endpoint accepts out-of-order payloads by timestamp — nonce replay protection still applies.
- The local compliance dashboard (on the Compliance Server itself, accessible only from the internal LAN) continues to function during cloud isolation with the locally stored findings.
- Implement a maximum queue depth: if the queue exceeds 7 days of payloads, raise a critical alert. This prevents unbounded local storage consumption during extended outages.

11. Glossary

Term	Definition
AES-256-GCM	Advanced Encryption Standard with 256-bit key in Galois/Counter Mode. Provides authenticated encryption — both confidentiality and tamper detection in a single operation.
CIS Benchmark	Center for Internet Security hardening guidelines. Level 2 is the configuration standard for systems handling sensitive data.
DCOM	Distributed Component Object Model. Microsoft protocol used by WMI for remote management connections.
Ed25519	An elliptic curve digital signature algorithm. Produces 64-byte signatures with a 128-bit security level. Faster and smaller than RSA-2048.
GGUF	A binary file format for storing quantised LLM model weights, used by llama.cpp. Q4_K_M refers to 4-bit quantisation with K-means clustering — balances quality and memory footprint.
GRC	Governance, Risk, and Compliance. The organisational framework for managing regulatory obligations, risk, and internal controls.
HSM	Hardware Security Module. A tamper-resistant physical device for generating and storing cryptographic keys. FIPS 140-2 Level 3 certified HSMs physically destroy keys if tampered with.
llama.cpp	An open-source C++ library for running quantised LLMs on CPU without requiring a GPU.
Nonce	Number used once. A random value included in each cryptographic payload to prevent replay attacks — a previously used nonce is rejected.
Qdrant	An open-source vector database. Stores numerical embeddings of text (framework controls) and performs fast nearest-neighbour search to find relevant controls during RAG.
RAG	Retrieval-Augmented Generation. A technique where the LLM's response is informed by relevant documents retrieved from a vector database, rather than relying solely on its training data.
Sentinel agent	The data collection module that connects to endpoints using read-only protocols (WMI or SSH) to harvest configuration data for compliance analysis.
TLS 1.3	Transport Layer Security version 1.3. The current standard for encrypted communications. Mandatory for this architecture — TLS 1.2 is not acceptable due to known weaknesses.
Transit secrets engine	HashiCorp Vault feature that performs cryptographic operations (signing, encryption) without exposing the key material — the key never leaves Vault.
WMI	Windows Management Instrumentation. Microsoft's standard interface for querying and managing Windows system configuration remotely.

END OF DOCUMENT — CONFIDENTIAL — FOR CISO REVIEW